

1 数据的表示和运算

1.1 定点数的编码表示

1. 正数：原码 == 反码 == 补码

2. 负数

- 原码 $\Leftrightarrow$ 反码

- ① 符号位保留，按位取反，末位加 1

- ② 符号位保留，最右 1 及右边保留，左边取反

- $[x]_{\text{补}} \Rightarrow [-x]_{\text{补}}$

- ① 按位取反，末位加 1

- ② 最右 1 及右边保留，左边取反

3. 移码：将补码的符号位取反

4. 机器数 $\Rightarrow$ 真值

- 权值法

- ① 8 位补码：11110110 $\Rightarrow -2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2^1 = -10$

- ② 16 位补码：8009H = 1000000000001001 $\Rightarrow -2^{15} + 2^3 + 2^0 = -32759$

- ③ 32 位补码：FFFF8009H 前面全是 F，说明：8009H 被符号扩展到了 32 位，所以真值和 8009H 相同

- 关键数

- ① 8000H = -32768, 8001H = -32767, 8002H = -32766

- ② 7FFFH = 32767

- ③ FFFFH = -1, FFFE H = -2, FFFCH = -4

1.2 整数的表示

1. 计算机中的有符号整数都用补码表示，所以 n 位有符号整数的表示范围 $-2^{n-1} \sim 2^{n-1} - 1$

1.2.1 C 语言中的整型数据类型

1. short 或 short int: 16bit

2. 整型 int: 32bit, 真值范围： $[-2^{31} \sim 2^{31} - 1] = [-2.147483648 * 10^9 \sim 2.147483647 * 10^9]$

3. long 或 long int: 32 位机器中 32 位，64 位机器中 64 位

4. 字符型 char: 8bit

1.2.2 不同字长整数之间的转换

1. 大字长 $\Rightarrow$ 小字长：高位部分直接截断，低位部分直接赋值
2. 小字长 $\Rightarrow$ 大字长：高位扩展
 - 无符号整数：零扩展
 - 有符号整数：符号扩展。符号扩展前后真值保持不变

1.3 运算方法

1. 与 ($\cdot$)，或 ($+$)，非 ($\overline{}$)，异或 ($\oplus$ ，输入相异则为 1)
2. 加法：补码直接相加。溢出：正 + 正 = 负 (上溢)、负 + 负 = 正 (下溢)
3. 减法：被减数与减数的负数补码相加。溢出：负 - 正 = 正、正 - 负 = 负
4. 乘法：左移一位乘以 2；加法。溢出判断
 - 有符号数：高 X 位的每一位都相同，且都等于低 X 位的符号，则表示不溢出，否则表示溢出
 - 无符号数：高 X 位全为 0，表示不溢出，否则表示溢出
5. 除法：右移一位除以 2；减法。
 - 溢出判断：若是 $2n$ 位除以 n 位的无符号数，商的位数为 $n + 1$ 位，当第一次试商为 1 时，则表示结果溢出
 - 若是两个 32 位 *int* 型整数相除，则除了 $\frac{\text{绝对值最大的负数}}{-1}$ 会溢出，其余情况都不会溢出
6. 无符号数和有符号数一起参与运算时，计算机按无符号数来解释最终的执行结果

1.3.1 移位运算

1. 逻辑移位 $\Rightarrow$ 无符号整数
 - 左移高位移出，低位补 0
 - 右移低位移出，高位补 0
 - 溢出：高位 1 移出，发生溢出
 - 丢失精度：低位 1 移出，丢失精度
2. 算术移位 $\Rightarrow$ 有符号整数
 - 左移高位移出，低位补 0
 - 右移低位移出，高位补符号位
 - 溢出：左移前后的符号位不同，发生溢出
 - 丢失精度：低位 1 移出，丢失精度

1.3.2 标志位

1. 零标志 ZF: $ZF = 1$ 表示结果为 0
2. 溢出标志 OF: 判断有符号数, $OF = C_n \oplus C_{n-1}$ (符号位进位与最高位进位的异或结果)
3. 符号标志 SF: 表示结果的符号
4. 进/错位标志 CF: 表示无符号数运算时的进位/借位
 - 加法: $CF = 1$ 结果溢出。最高位是否产生进位
 - 减法: $CF = 1$ 表示有借位, 即不够减

1.4 浮点数的表示

1.4.1 理论浮点数

1. 原码尾数规格化
 - 正数: $0.1xxx$
 - 负数: $1.1xxx$
2. 补码尾数规格化: 符号位与最高数值位必须相反。
 - $+0.75 \Rightarrow 0.11$
 - $-0.75 \Rightarrow 1.01$

1.4.2 IEEE 754 标准

		单精度	双精度
符号 s		1bit	1bit
阶码 e		8bit(范围: [1, 254])	11bit(范围: [1, 2046])
尾数 f (尾数有隐藏位 1)		23bit	52bit
总位数		32bit (原码)	64bit (原码)
偏置值		127(7FH)	1023(3FFH)
指数		阶码真值 + 偏置值, 后按无符号整数规则转换	
真值		$(-1)^s \times 1.f \times 2^{e-127}$	$(-1)^s \times 1.f \times 2^{e-1023}$
最小值		$e = 1, f = 0 \Rightarrow 1.0 * 2^{1-127} = 2^{-126}$	$e = 1, f = 0 \Rightarrow 1.0 * 2^{1-1023} = 2^{-1022}$
最大值		$e = 254, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{254-127} = 2^{127} * (2 - 2^{-23})$	$e = 2046, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{2046-1023} = 2^{1023} * (2 - 2^{-52})$
阶码全 1	尾数全 0	$\pm\infty$	
	尾数非 0	NaN	
阶码全 0	尾数全 0	± 0	
	尾数非 0	非规格化数 (隐藏位为 0)	
判断溢出		规格化后阶码超出所能表示的范围时, 发生溢出	

1. 真值 $\Rightarrow$ IEEE 754 浮点数

- ① 确定符号位
- ② 真值转化为二进制
- ③ 规格化确定尾数和阶码真值
 - 左规: 尾数每左移一位, 阶码减 1
 - 右规: 尾数每右移一位, 阶码加 1
- ④ 阶码真值 + 偏置值, 后按无符号整数规则转换

1.4.3 加减运算

1. 对阶: 使两个操作数的小数点对齐

- 小阶码向大阶码看齐
- IEEE 规定右移出的 *bit* 至少额外保留 3 位

2. 尾数加减

3. 尾数规格化

4. 舍入

- 就近舍入：0 舍 1 入。特殊情况：3 个舍弃位是 100
 - ① $1 + 23bit$ 尾数末位 0 $\Rightarrow$ 截断
 - ② $1 + 23bit$ 尾数末位 1 $\Rightarrow$ 截断多余位，并末位加 1
- 场景
 - ① 对阶
 - ② 右规格化

5. 溢出判断

- 一个正指数超过了最大允许值 (127 或 1023) $\Rightarrow$ 指数上溢 (阶码 11111111)，将尾数强制保存为全 0
- 一个负指数超过了最小允许值 (-149 或 -1074) $\Rightarrow$ 指数下溢 (若当前阶码 00000001，尾数仍需左规，直接将阶码置为全 0，并且尾数不移位)

1.4.4 C 语言中的浮点数类型

1. 不同类型数的混合运算，遵循的原则是类型提升，即较低类型转换为较高类型
2. *float* : 4Byte, 真值范围: $[-3.4 * 10^{38} \sim 3.4 * 10^{38}]$
3. *double* : 8Byte, 真值范围: $[-1.79 * 10^{308} \sim 1.79 * 10^{308}]$
4. *int* $\Rightarrow$ *float*: 可能精度丢失
5. *double* $\Rightarrow$ *float*: 可能溢出或精度丢失
6. *float/double* $\Rightarrow$ *int*: 仅保留整数部分，可能溢出